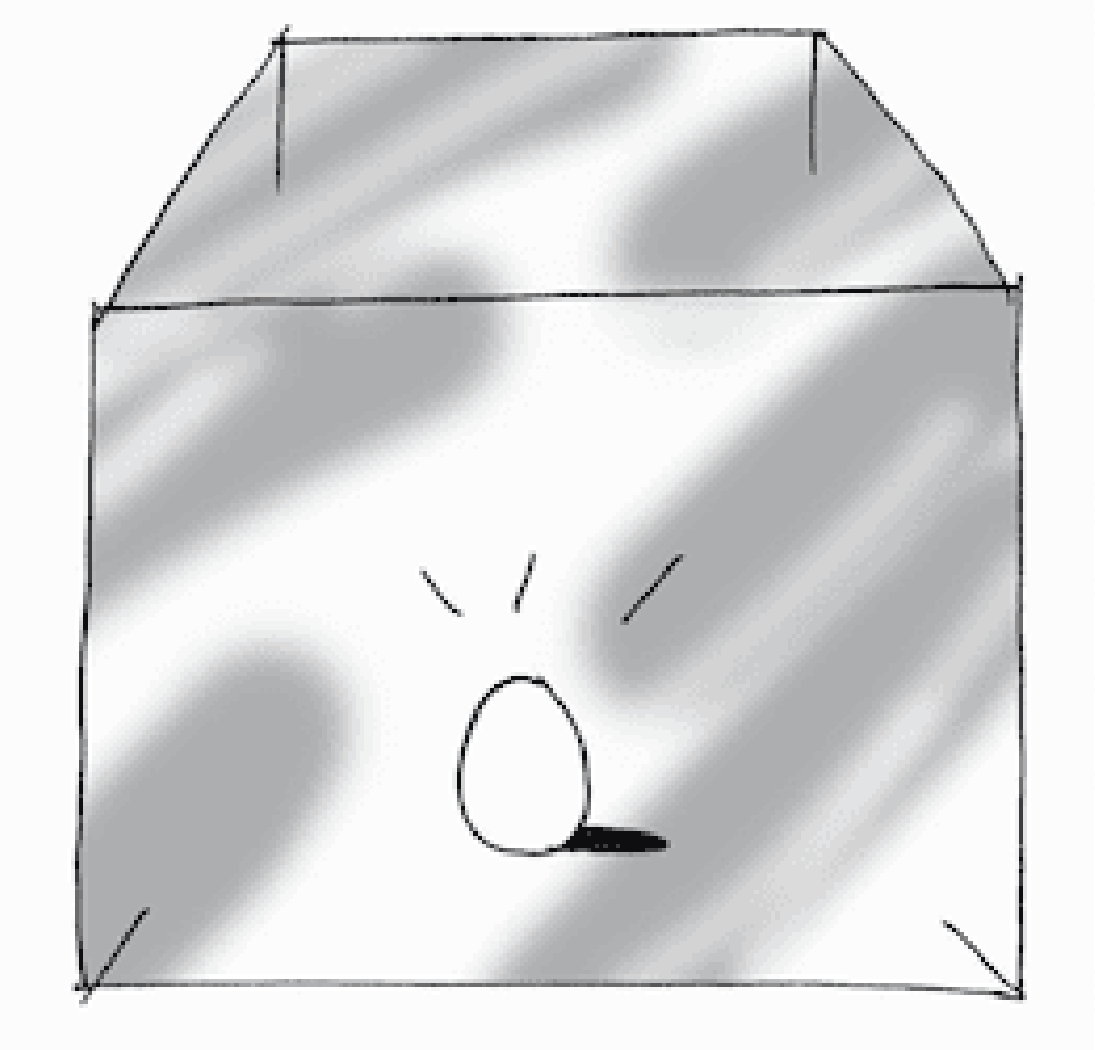
Singleton Pattern

Only One Instance



Singleton Pattern

When every object holds the same data, creating multiple copies wastes memory and effort.

Why not keep a single shared instance and reuse it everywhere?

This saves resources and ensures consistency.

The Problem

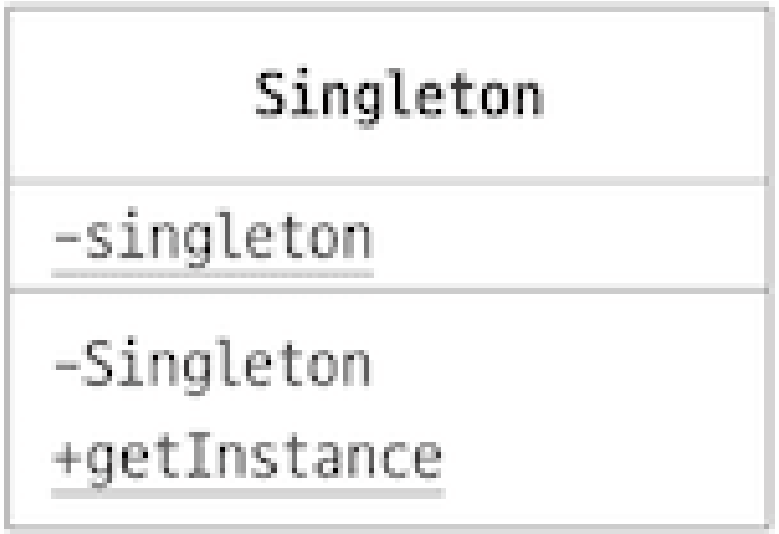
- Sometimes we need exactly one instance of a class in the entire application.
- Creating multiple instances could cause problems or waste resources.
- Database connection, logger, configuration settings needs only one instance.

The challenge: how to ensure a class has only one instance and provide global access to it?

The *Singleton* as the Solution

- Ensure a class has only one instance and provide a global point of access to it.
- The class itself is responsible for keeping track of its sole instance.
- The class can ensure that no other instances can be created.

The Design



Step 1: Understand the Players

In this design, we have only one player:

- **Singleton** (ensures only one instance exists)

Step 2: Key Characteristics

- **Single stored instance**: The class keeps the one shared object.
- **Class method**: Provides the access point.
- **Lazy Initialization**: Instance created when first needed

Code

- Main Method
- Singleton Implementation

Main Method

```
from singleton import Singleton

def main():
    print("=== Singleton Pattern Example ===\n")

    obj1 = Singleton.get_instance(100)
    obj2 = Singleton.get_instance(200)  # 200 is ignored

    if obj1 is obj2:
        print("obj1, obj2 are the same instances.")

    print(f"obj1: {obj1.value}")  # 100
    print(f"obj2: {obj2.value}")  # 100
```

Step 1: Create multiple "instances"

```
obj1 = Singleton.get_instance(100)
obj2 = Singleton.get_instance(200)
```

- All variables point to the same instance.
- Only the first value (100) is used for initialization.

Step 2: Verify they are the same

```
if obj1 is obj2:
    print("obj1, obj2 are the same instances.")
```

- The `is` operator checks object identity.
- Both variables refer to the same object in memory.

Output Example

```
An instance is created.
obj1 and obj2 are the same instances.
obj1: 100
obj2: 100
```

Singleton Implementation

```
class Singleton:
    _instance = None
    def __init__(self, value):
        self.value = value

    @classmethod
    def get_instance(cls, value=None):
        if cls._instance is None:
            print("An instance is created.")
            cls._instance = cls(value)
        return cls._instance
```

- `get_instance()` is the creation path used by the client.

Discussion

Consider DIP

A Good alternative to Singleton pattern in modern programming is Dependency Injection:

Dependency Injection - instead of global access, explicitly pass dependencies through constructors or methods. This makes dependencies visible and testing easier.

Use static method

```
# Bypasses the intended access path
s1 = Singleton(1)
s2 = Singleton(2)
print(s1 is s2)  # False

# Right way
s1 = Singleton.get_instance()
s2 = Singleton.get_instance()
print(s1 is s2)  # True
```

Singleton Benefits and Drawbacks

Benefits:

- **Controlled Access**: Single point of access
- **Reduced Memory**: Only one instance
- **Global State**: Shared across application

Drawbacks:

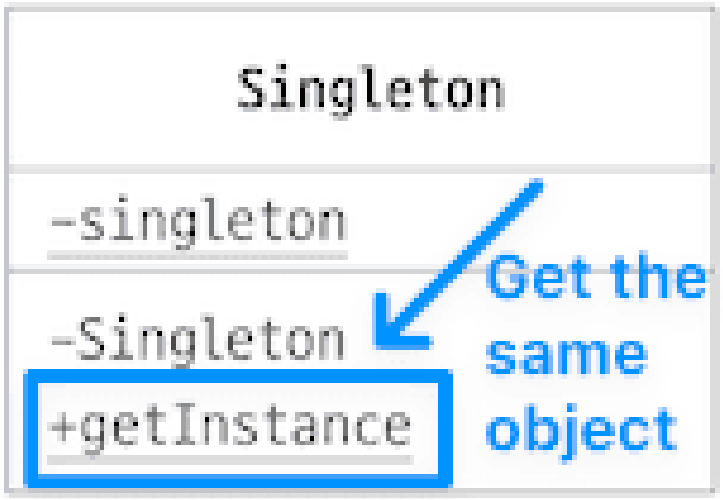
- **Testing Difficulty**: Hard to mock or reset
- **Hidden Dependencies**: Global state can cause issues
- **Thread Safety**: Need careful implementation in

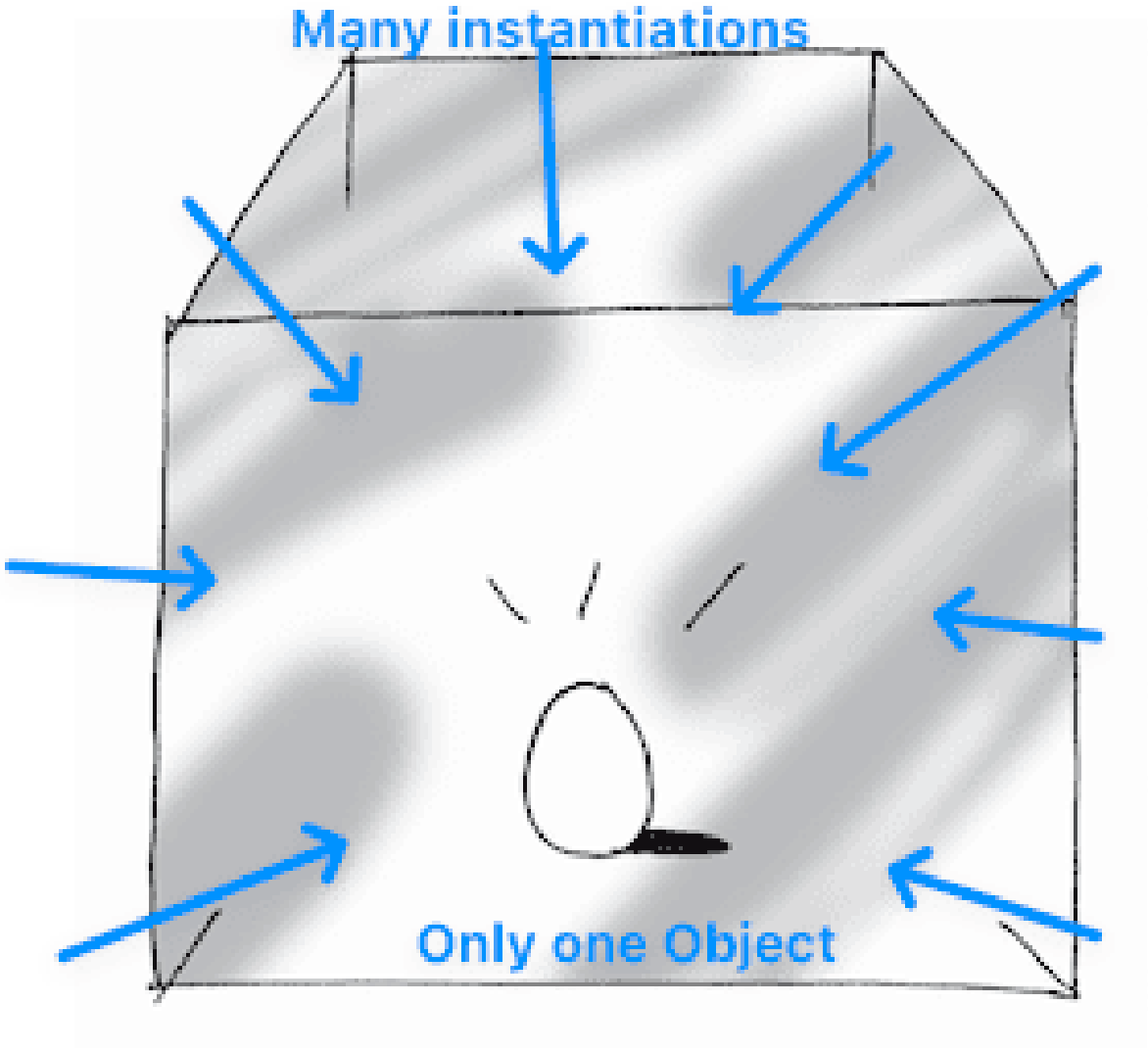
Related Patterns

Factory Method: Factory can be implemented as a Singleton

For example, instead of multiple factories creating DatabaseConnection objects, you enforce a single factory object (singleton) that ensures all connections come from a single source.

UML


Singleton
-singleton
-Singleton
+getInstance
Get the same object


Many instantiations
Only one Object